

Matplotlib

Clase 14

count()

Retorna la cantidad de elementos no nulos en la Serie o en cada columna del DataFrame. Si aplicas count() a un DataFrame, se calculará el número de elementos no nulos en cada columna del DataFrame.

```
import pandas as pd
# Crear una Serie con valores nulos
data = [1, 2, None, 4, None, 6]
series = pd.Series(data)

# Contar elementos no nulos
count_result = series.count()
print(count_result) # Salida: 4
```

```
import pandas as pd
# Crear un DataFrame con valores nulos
data = {'A': [1, 2, None], 'B': [4, None, 6]}
df = pd.DataFrame(data)

# Aplicar count a cada columna del DataFrame
count_result = df.count()
print(count_result)
# Salida:
# A    2
# B    2
# dtype: int64
```

size

Retorna la cantidad total de elementos en la Serie o el número total de celdas en el DataFrame (incluidas las celdas nulas).

```
import pandas as pd

# Crear un DataFrame con valores nulos
data = {'A': [1, 2, None], 'B': [4, None, 6]}
df = pd.DataFrame(data)

# Obtener el tamaño total del DataFrame
size_result = df.size
print(size_result) # Salida: 6
```

value_counts()

Retorna una Serie que contiene recuentos de valores únicos. Muestra la frecuencia de cada valor en la Serie. Si aplicas value_counts() a un DataFrame, obtendrás un error porque este método está diseñado para Series, no para DataFrames.

```
# Crear una Serie con valores repetidos
data = ['a', 'b', 'a', 'c', 'b', 'a']
series = pd.Series(data)

# Contar la frecuencia de cada valor único
value_counts_result = series.value_counts()
print(value_counts_result)

# Salida:
# a    3
# b    2
# c    1
```

Algunas funciones en pandas

- **fillna:** se utiliza en Pandas para rellenar valores faltantes (NaN) en un DataFrame o Serie con un valor específico.

```
import pandas as pd
```

```
# Crear un DataFrame con valores faltantes
```

```
data = {'A': [1, 2, None, 4], 'B': [None, 6, 7, 8]}
```

```
df = pd.DataFrame(data)
```

```
# Rellenar los valores faltantes con cero
```

```
df_filled = df.fillna(0)
```

```
print(df_filled)
```

Algunas funciones en pandas

- **str:** se utiliza para acceder a métodos de manipulación de cadenas de caracteres en una Serie de Pandas que contenga datos de tipo str.

```
import pandas as pd
```

```
# Crear una Serie con datos de tipo string
```

```
data = {'nombres': ['Juan', 'María', 'Pedro', 'Luis']}
```

```
serie = pd.Series(data['nombres'])
```

```
# Convertir todos los nombres a mayúsculas
```

```
serie_upper = serie.str.upper()
```

```
print(serie_upper)
```

Algunas funciones en pandas

- **apply:** se utiliza para aplicar una función a lo largo de un eje (filas o columnas) de un DataFrame o Serie.

```
import pandas as pd
```

```
# Crear un DataFrame
```

```
data = {'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8]}
```

```
df = pd.DataFrame(data)
```

```
# Definir una función que multiplique cada valor por 2
```

```
def multiply_by_2(x):
```

```
    return x * 2
```

```
# Aplicar la función a todas las columnas del DataFrame
```

```
df_applied = df.apply(multiply_by_2)
```

```
print(df_applied)
```

Matplotlib

- Matplotlib es una biblioteca popular en Python para crear gráficos y visualizaciones.
- Proporciona una amplia gama de opciones para representar datos en forma de gráficos de alta calidad.
- Es flexible y se puede utilizar en una variedad de aplicaciones, desde análisis de datos hasta generación de informes y presentaciones.



Gráfico de Línea

Datos Estructurados:

- Se espera que los datos estén en forma de pares (x, y), donde "x" representa la variable independiente y "y" representa la variable dependiente.
- Puede haber múltiples líneas en el mismo gráfico, cada una representando una serie de datos diferente.

Uso Típico:

Ideal para mostrar tendencias a lo largo del tiempo o en una secuencia ordenada.

Ejemplos Líneas

```
import matplotlib.pyplot as plt
```

```
# Datos de ejemplo
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [10, 15, 8, 12, 6]
```

```
# Crear gráfico de línea
```

```
plt.plot(x, y)
```

```
# Personalizar el gráfico
```

```
plt.title('Ejemplo de Gráfico de Línea')
```

```
plt.xlabel('Eje X')
```

```
plt.ylabel('Eje Y')
```

```
# Mostrar el gráfico
```

```
plt.show()
```

```
import matplotlib.pyplot as plt
```

```
x = [1, 2, 3, 4, 5]
```

```
y1 = [10, 15, 7, 12, 20]
```

```
y2 = [5, 8, 3, 6, 10]
```

```
plt.plot(x, y1, label='Serie 1')
```

```
plt.plot(x, y2, label='Serie 2')
```

```
plt.xlabel('Eje X')
```

```
plt.ylabel('Eje Y')
```

```
plt.legend()
```

```
plt.show()
```

Gráfico de Barras

Datos Estructurados:

- Se espera que los datos estén en forma de categorías y sus valores asociados.
- Puede haber barras agrupadas o apiladas para comparar múltiples conjuntos de datos.

Uso Típico:

Bueno para comparar cantidades entre diferentes categorías.

Ejemplos Barras

```
import matplotlib.pyplot as plt
```

```
# Datos de ejemplo
```

```
categorias = ['A', 'B', 'C', 'D', 'E']
```

```
valores = [15, 20, 10, 25, 12]
```

```
# Crear gráfico de barras
```

```
plt.bar(categorias, valores)
```

```
# Personalizar el gráfico
```

```
plt.title('Ejemplo de Gráfico de Barras')
```

```
plt.xlabel('Categorías')
```

```
plt.ylabel('Valores')
```

```
# Mostrar el gráfico
```

```
plt.show()
```

```
import matplotlib.pyplot as plt
```

```
categorias = ['A', 'B', 'C', 'D', 'E']
```

```
valores_serie1 = [4, 7, 1, 5, 10]
```

```
valores_serie2 = [6, 2, 8, 3, 5]
```

```
plt.bar(categorias, valores_serie1, label='Serie 1')
```

```
plt.bar(categorias, valores_serie2,  
bottom=valores_serie1, label='Serie 2')
```

```
plt.xlabel('Categorías')
```

```
plt.ylabel('Valores')
```

```
plt.legend()
```

```
plt.show()
```

Gráfico de Torta

- Datos Estructurados: Se espera que los datos estén en forma de porcentajes o fracciones que suman 100%.
- Uso Típico: Ideal para mostrar la proporción de cada parte con respecto al todo.

```
# Datos de ejemplo
```

```
etiquetas = ['A', 'B', 'C', 'D']
```

```
valores = [30, 20, 15, 35]
```

```
# Crear gráfico de torta
```

```
plt.pie(valores, labels=categorias, autopct='%1.1f%%')
```

```
# Personalizar el gráfico
```

```
plt.title('Ejemplo de Gráfico de Torta')
```

```
# Mostrar el gráfico
```

```
plt.show()
```

Guardar una imagen

- Matplotlib permite guardar los gráficos creados en diferentes formatos de archivo, como PNG, JPEG, PDF, SVG, entre otros.
- Esto es útil para compartir los gráficos, incluirlos en informes o presentaciones, o utilizarlos en otros contextos.
- Se pueden especificar la ubicación y el nombre del archivo al guardar el gráfico

```
# Guarda la imagen en un archivo png  
plt.savefig('grafico.png')
```

```
plt.show()
```

Ejercicio 1

Limpiar el siguiente data frame, rellenando las edades vacías con la edad promedio. Los emails vacíos deben de decir 'desconocido'. Y los la primera letra de los nombres deben estar en mayúscula.

```
data = {  
    'Nombre': ['juan', 'maría', None, 'luis'],  
    'Edad': [25, None, 30, 40],  
    'Ciudad': ['Barcelona', 'Madrid', 'Barcelona', 'Valencia'],  
    'Email': ['juan@example.com', 'maria@example.com', 'pedro@example.com', None]  
}
```

Ejercicio 2

Mostrar en un gráfico de barras, las ganancias por año sin descuento del ejercicio 1 de la clase pasada.

Ejercicio 3

Mostrar y guardar un gráfico de torta donde se vean el porcentaje total de películas versus series que hay en el data set.

Ejercicio 4

Mostrar y guardar en un gráfico de líneas la evolución de la cantidad de producciones estrenadas a lo largo de los años.